Abdelrahman Khalifa

Open-Source Software Engineer | Rust, C, Linux

abdelrahman.khalifa.gad@gmail.com | +20 114 250 5042 | Cairo, Egypt

linkedin.com/in/abd002 | github.com/Abd002 | codeforces.com/profile/_Khalifa

SUMMARY

Software engineer with **14 merged upstream PRs since March 2025**: 11 in OpenPrinting (libcups, the CUPS library in C, and its Rust bindings cups-rs) as a Google Summer of Code 2026 contributor with The Linux Foundation (OpenPrinting), plus Rust Clippy, Boa and Zephyr RTOS. Every change went through maintainer code review. Comfortable building and testing C and Rust projects locally and in CI against real services, and explaining changes clearly in writing.

EXPERIENCE

Google Summer of Code Contributor | **The Linux Foundation (OpenPrinting)** | May 2026 – Oct 2026

Open-source project: Printers page for COSMIC Settings, in Rust; passed the final evaluation | *github.com/Abd002/GSOC-2026* | *github.com/Abd002/cosmic-printers*

- Built the **Printers page for COSMIC Settings** and a standalone app sharing the same UI: printers grouped by device, status and supply levels, print queue control, test pages, live updates, and an Add Printer flow that sets up legacy printers through Printer Applications (PAPPL) instead of PPD drivers (cosmic-settings #2182, in review).
- Designed the stack as 5 Rust crates: `printers-core` (types, grouping queues into physical devices), `printers-client` (async varlink client), `printers-server` (CUPS/IPP), `printers-ui` (iced) and `printers-app`.
- Ran the backend as a varlink IPC service in `cosmic-settings-daemon` (cosmic-settings-daemon #188, in review), so the C FFI code never loads in the Settings process; the standalone app falls back to running it in-process.
- Got **11 PRs merged upstream** in OpenPrinting (8 during the program). cups-rs (Rust): ported it to libcups3 with CUPS 2 and CUPS 3 builds and a CI matrix across architectures; added IPP management operations, Printer Application support, IP/port connections, `lpoptions` and FFI bindings for the libcups3 DNS-SD API (9 PRs). libcups (C): fixed crashes in `cupsDNSSDDelete()` and a leaked client in the Avahi DNS-SD code (#164, #165).
- Tested on real HP printers and Printer Applications (Ghostscript, Gutenprint, HPLIP); CI checks formatting, Clippy and tests on several architectures and runs CUPS, Avahi and `ippeveprinter`.

Embedded Software Engineer | **EKSON** | Aug 2025 – Present

Software for 3-DoF and 6-DoF pneumatic motion simulators

- Wrote the ESP32-S3 firmware and migrated it from ESP-IDF/FreeRTOS to Zephyr RTOS: PID valve control with IMU feedback, safety stops and a UART protocol.
- Developed the PyQt5 desktop control app, cutting CPU usage and latency by about 40% (measured) using asyncio to wait for ESP32 and server messages concurrently; added remote control and firmware updates through a Node.js/MongoDB backend.

Embedded Linux Systems Intern | **Orange Digital Center (ODC)** | Dec 2024 – Jan 2025

- Created custom Linux images with Yocto and Buildroot and wrote a Linux kernel module; worked with device trees, root file systems and hardware flashing. Scored 100% in the final evaluation.

OTHER OPEN-SOURCE CONTRIBUTIONS

Contributor (all merged) — Zephyr RTOS, Boa JavaScript engine (Rust), Rust Clippy | Mar 2025 – Dec 2025

- Zephyr RTOS: added the `IP_MULTICAST_LOOP` socket option for IPv4 in 6 networking modules (#86977).
- Boa: optimized `JsStr::code_points()` for Latin1 strings, avoiding UTF-16 conversion (PR #4553).
- Rust Clippy: fixed `ref_as_ptr` false positives for non-temporary references in `let/const` (PR #16214).

PROJECTS

- **Kernel Monitor** (C, Linux, ARM, QEMU): character device driver that walks the process list under RCU and exports per-task IDs, memory, threads, context switches and CPU time to user space; run on QEMU. GitHub
- **Database Systems, CMU 15-445** (C++17): thread-safe Count-Min Sketch for the BusTub database (one mutex per row; insert, count, merge, top-k) and SQL analytics in DuckDB. GitHub

TECHNICAL SKILLS

Languages: Rust, C, Modern C++, Python (asyncio), Bash, JavaScript (Node.js)

Systems & Linux: Linux (Ubuntu/Debian), operating systems, FFI and bindings, IPC (varlink, D-Bus; zbus, zlink), concurrency (async/event-driven, multithreading), memory management, networking (TCP/UDP), CUPS/IPP, DNS-SD/mDNS (Avahi), kernel modules and device drivers

Tooling: Code review, testing, Git/GitHub, GitHub Actions CI (multi-architecture matrices), Cargo, Clippy, rustfmt, CMake, Make, Yocto, Buildroot, QEMU, debugging, performance optimization

Desktop & embedded: iced, libcosmic, PyQt5; Zephyr, FreeRTOS, ARM Cortex-M (STM32), ESP32

EDUCATION

Bachelor of Science (B.Sc.), Computer and Systems Engineering | **Minia University** | 2020 – 2025

- Grade: Very Good with Honors. Graduation project: AUTOSAR V2X vehicle on STM32 with a dual-bank bootloader.
- Activities: Codeforces Expert; 4-time ECPC finalist (2021–2024); Treasurer, IEEE Student Branch.